

Studentai

2.0

Generated by Doxygen 1.17.0

1 Hierarchical Index	1
1.1 Class Hierarchy	1
2 Class Index	3
2.1 Class List	3
3 File Index	5
3.1 File List	5
4 Class Documentation	7
4.1 Studentas Class Reference	7
4.1.1 Detailed Description	8
4.1.2 Constructor & Destructor Documentation	8
4.1.2.1 Studentas() [1/3]	8
4.1.2.2 ~Studentas()	9
4.1.2.3 Studentas() [2/3]	9
4.1.2.4 Studentas() [3/3]	9
4.1.3 Member Function Documentation	9
4.1.3.1 addPaz()	9
4.1.3.2 clearPaz()	9
4.1.3.3 getEgz()	9
4.1.3.4 getMedrez()	10
4.1.3.5 getPaz()	10
4.1.3.6 getRez()	10
4.1.3.7 info()	10
4.1.3.8 operator=() [1/2]	10
4.1.3.9 operator=() [2/2]	10
4.1.3.10 refEgz()	11
4.1.3.11 refMedrez()	11
4.1.3.12 refPaz()	11
4.1.3.13 refRez()	11
4.1.3.14 setEgz()	11
4.1.3.15 skaiciuoti()	11
4.2 Zmogus Class Reference	12
4.2.1 Detailed Description	12
4.2.2 Constructor & Destructor Documentation	13
4.2.2.1 Zmogus() [1/3]	13
4.2.2.2 ~Zmogus()	13
4.2.2.3 Zmogus() [2/3]	13
4.2.2.4 Zmogus() [3/3]	13
4.2.3 Member Function Documentation	13
4.2.3.1 getPavarde()	13
4.2.3.2 getVardas()	13

4.2.3.3 info()	14
4.2.3.4 operator=() [1/2]	14
4.2.3.5 operator=() [2/2]	14
4.2.3.6 refPavarde()	14
4.2.3.7 refVardas()	14
4.2.3.8 setPavarde()	14
4.2.3.9 setVardas()	14
4.2.4 Member Data Documentation	15
4.2.4.1 pavarde	15
4.2.4.2 vardas	15
5 File Documentation	17
5.1 funk.cpp File Reference	17
5.1.1 Function Documentation	18
5.1.1.1 cmpEgz()	18
5.1.1.2 cmpMedrez()	18
5.1.1.3 cmpPavarde()	19
5.1.1.4 cmpRez()	19
5.1.1.5 cmpVardas()	19
5.1.1.6 failgeneravimas()	19
5.1.1.7 failinputas()	19
5.1.1.8 failoutputas()	20
5.1.1.9 genirasymas()	20
5.1.1.10 inputas()	20
5.1.1.11 logResults()	20
5.1.1.12 lytgen()	20
5.1.1.13 operator<<()	21
5.1.1.14 operator>>()	21
5.1.1.15 outputas()	21
5.1.1.16 randominputas()	21
5.1.1.17 randompavarde()	21
5.1.1.18 randomvardas()	22
5.1.1.19 rikiuoti()	22
5.1.1.20 skirstyti()	22
5.1.1.21 skirstyti2()	22
5.1.1.22 skirstyti3()	22
5.2 funk.cpp	23
5.3 studentai.h File Reference	30
5.3.1 Enumeration Type Documentation	31
5.3.1.1 SkirstymoStrategija	31
5.3.2 Function Documentation	32
5.3.2.1 cmpEgz()	32

5.3.2.2 cmpMedrez()	32
5.3.2.3 cmpPavarde()	32
5.3.2.4 cmpRez()	32
5.3.2.5 cmpVardas()	33
5.3.2.6 failgeneravimas()	33
5.3.2.7 failinputas()	33
5.3.2.8 failoutputas()	33
5.3.2.9 genirasymas()	33
5.3.2.10 inputas()	34
5.3.2.11 logResults()	34
5.3.2.12 lytgen()	34
5.3.2.13 operator<<()	34
5.3.2.14 operator>>()	34
5.3.2.15 outputas()	35
5.3.2.16 randominputas()	35
5.3.2.17 randompavarde()	35
5.3.2.18 randomvardas()	35
5.3.2.19 rikiuoti()	35
5.3.2.20 skirstyti()	36
5.3.2.21 skirstyti2()	36
5.3.2.22 skirstyti3()	36
5.3.3 Variable Documentation	36
5.3.3.1 fgalunes	36
5.3.3.2 fvardai	36
5.3.3.3 irasymo_failas	36
5.3.3.4 irasymo_failas_blogas	37
5.3.3.5 irasymo_failas_geras	37
5.3.3.6 mgalunes	37
5.3.3.7 mvardai	37
5.3.3.8 pavardes	37
5.4 studentai.h	37
5.5 versija_v2.0.cpp File Reference	39
5.5.1 Function Documentation	40
5.5.1.1 main()	40
5.6 versija_v2.0.cpp	40

Chapter 1

Hierarchical Index

1.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

Zmogus	12
Studentas	7

Chapter 2

Class Index

2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Studentas		
Studentas	Studentas klasė, paveldi iš Zmogus	7
Zmogus		
	Bazinė abstrakti klasė Zmogus	12

Chapter 3

File Index

3.1 File List

Here is a list of all files with brief descriptions:

funk.cpp	17
studentai.h	30
versija_v2.0.cpp	39

Chapter 4

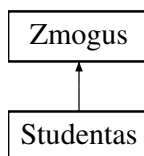
Class Documentation

4.1 Studentas Class Reference

`Studentas` klasė, paveldi iš `Zmogus`.

```
#include <studentai.h>
```

Inheritance diagram for `Studentas`:



Public Member Functions

- `Studentas` (const string &v="A", const string &p="BB", const vector< int > &paz_={}, int egz_=0)
Konstruktorius su pažymiais ir egzaminu.
- `~Studentas` ()
- `Studentas` (const `Studentas` &other)
Kopijavimo konstruktorius.
- `Studentas` & `operator=` (const `Studentas` &other)
Kopijavimo operatorius.
- `Studentas` (`Studentas` &&other) noexcept
move konstruktorius
- `Studentas` & `operator=` (`Studentas` &&other) noexcept
move operatorius
- void `info` () const override
Abstrakti informacijos funkcija.
- vector< int > & `refPaz` ()
- int & `refEgz` ()
- double & `refRez` ()
- double & `refMedrez` ()
- void `clearPaz` ()
- double `getRez` () const

- double [getMedrez](#) () const
- int [getEgz](#) () const
- const vector< int > & [getPaz](#) () const
- void [addPaz](#) (int p)
- void [setEgz](#) (int e)
- void [skaiciuoti](#) ()

apskaičiuoja galutinius medianinius ir vidurkinius studento rezultatus

Public Member Functions inherited from [Zmogus](#)

- [Zmogus](#) (const string &v="A", const string &p="BB")
Bazinis konstruktorius.
- virtual [~Zmogus](#) ()
Virtualus destruktorius.
- string & [refVardas](#) ()
- string & [refPavarde](#) ()
- string [getVardas](#) () const
- string [getPavarde](#) () const
- void [setVardas](#) (const string &v)
- void [setPavarde](#) (const string &p)
- [Zmogus](#) (const [Zmogus](#) &)=default
Rule of Zero (default) copy/move.
- [Zmogus](#) & [operator=](#) (const [Zmogus](#) &)=default
- [Zmogus](#) ([Zmogus](#) &&) noexcept=default
- [Zmogus](#) & [operator=](#) ([Zmogus](#) &&) noexcept=default

Additional Inherited Members

Protected Attributes inherited from [Zmogus](#)

- string [vardas](#)
- string [pavarde](#)

4.1.1 Detailed Description

[Studentas](#) klasė, paveldi iš [Zmogus](#).

Definition at line 55 of file [studentai.h](#).

4.1.2 Constructor & Destructor Documentation

4.1.2.1 Studentas() [1/3]

```
Studentas::Studentas (
    const string & v = "A",
    const string & p = "BB",
    const vector< int > & paz_ = {},
    int egz_ = 0) [inline]
```

Konstruktorius su pažymiais ir egzaminu.

Definition at line 64 of file [studentai.h](#).

4.1.2.2 ~Studentas()

```
Studentas::~~Studentas () [inline]
```

Definition at line 81 of file [studentai.h](#).

4.1.2.3 Studentas() [2/3]

```
Studentas::Studentas (  
    const Studentas & other) [inline]
```

Kopijavimo konstruktorius.

Definition at line 88 of file [studentai.h](#).

4.1.2.4 Studentas() [3/3]

```
Studentas::Studentas (  
    Studentas && other) [inline], [noexcept]
```

move konstruktorius

Definition at line 109 of file [studentai.h](#).

4.1.3 Member Function Documentation

4.1.3.1 addPaz()

```
void Studentas::addPaz (  
    int p) [inline]
```

Definition at line 152 of file [studentai.h](#).

4.1.3.2 clearPaz()

```
void Studentas::clearPaz () [inline]
```

Definition at line 145 of file [studentai.h](#).

4.1.3.3 getEgz()

```
int Studentas::getEgz () const [inline]
```

Definition at line 148 of file [studentai.h](#).

4.1.3.4 getMedrez()

```
double Studentas::getMedrez () const [inline]
```

Definition at line 147 of file [studentai.h](#).

4.1.3.5 getPaz()

```
const vector< int > & Studentas::getPaz () const [inline]
```

Definition at line 150 of file [studentai.h](#).

4.1.3.6 getRez()

```
double Studentas::getRez () const [inline]
```

Definition at line 146 of file [studentai.h](#).

4.1.3.7 info()

```
void Studentas::info () const [inline], [override], [virtual]
```

Abstrakti informacijos funkcija.

Implements [Zmogus](#).

Definition at line 137 of file [studentai.h](#).

4.1.3.8 operator=() [1/2]

```
Studentas & Studentas::operator= (  
    const Studentas & other) [inline]
```

Kopijavimo operatorius.

Definition at line 97 of file [studentai.h](#).

4.1.3.9 operator=() [2/2]

```
Studentas & Studentas::operator= (  
    Studentas && other) [inline], [noexcept]
```

move operatorius

Definition at line 122 of file [studentai.h](#).

4.1.3.10 refEgz()

```
int & Studentas::refEgz () [inline]
```

Definition at line 142 of file [studentai.h](#).

4.1.3.11 refMedrez()

```
double & Studentas::refMedrez () [inline]
```

Definition at line 144 of file [studentai.h](#).

4.1.3.12 refPaz()

```
vector< int > & Studentas::refPaz () [inline]
```

Definition at line 141 of file [studentai.h](#).

4.1.3.13 refRez()

```
double & Studentas::refRez () [inline]
```

Definition at line 143 of file [studentai.h](#).

4.1.3.14 setEgz()

```
void Studentas::setEgz (  
    int e) [inline]
```

Definition at line 153 of file [studentai.h](#).

4.1.3.15 skaiciuoti()

```
void Studentas::skaiciuoti ()
```

apskaičiuoja galutinius medianinius ir vidurkinius studento rezultatus

Compute final average and median-based results

Definition at line 17 of file [funk.cpp](#).

The documentation for this class was generated from the following files:

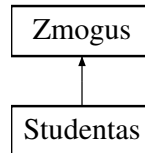
- [studentai.h](#)
- [funk.cpp](#)

4.2 Zmogus Class Reference

Bazinė abstrakti klasė [Zmogus](#).

```
#include <studentai.h>
```

Inheritance diagram for Zmogus:



Public Member Functions

- [Zmogus](#) (const string &v="A", const string &p="BB")
Bazinis konstruktorius.
- virtual [~Zmogus](#) ()
Virtualus destruktorius.
- virtual void [info](#) () const =0
Abstrakti informacijos funkcija.
- string & [refVardas](#) ()
- string & [refPavarde](#) ()
- string [getVardas](#) () const
- string [getPavarde](#) () const
- void [setVardas](#) (const string &v)
- void [setPavarde](#) (const string &p)
- [Zmogus](#) (const [Zmogus](#) &)=default
Rule of Zero (default) copy/move.
- [Zmogus](#) & [operator=](#) (const [Zmogus](#) &)=default
- [Zmogus](#) ([Zmogus](#) &&) noexcept=default
- [Zmogus](#) & [operator=](#) ([Zmogus](#) &&) noexcept=default

Protected Attributes

- string [vardas](#)
- string [pavarde](#)

4.2.1 Detailed Description

Bazinė abstrakti klasė [Zmogus](#).

Definition at line 24 of file [studentai.h](#).

4.2.2 Constructor & Destructor Documentation

4.2.2.1 Zmogus() [1/3]

```
Zmogus::Zmogus (
    const string & v = "A",
    const string & p = "BB") [inline]
```

Bazinis konstruktorius.

Definition at line 31 of file [studentai.h](#).

4.2.2.2 ~Zmogus()

```
virtual Zmogus::~Zmogus () [inline], [virtual]
```

Virtualus destruktoriaus.

Definition at line 35 of file [studentai.h](#).

4.2.2.3 Zmogus() [2/3]

```
Zmogus::Zmogus (
    const Zmogus & ) [default]
```

Rule of Zero (default) copy/move.

4.2.2.4 Zmogus() [3/3]

```
Zmogus::Zmogus (
    Zmogus && ) [default], [noexcept]
```

4.2.3 Member Function Documentation

4.2.3.1 getPavarde()

```
string Zmogus::getPavarde () const [inline]
```

Definition at line 43 of file [studentai.h](#).

4.2.3.2 getVardas()

```
string Zmogus::getVardas () const [inline]
```

Definition at line 42 of file [studentai.h](#).

4.2.3.3 info()

```
virtual void Zmogus::info () const [pure virtual]
```

Abstrakti informacijos funkcija.

Implemented in [Studentas](#).

4.2.3.4 operator=() [1/2]

```
Zmogus & Zmogus::operator= (
    const Zmogus & ) [default]
```

4.2.3.5 operator=() [2/2]

```
Zmogus & Zmogus::operator= (
    Zmogus && ) [default], [noexcept]
```

4.2.3.6 refPavarde()

```
string & Zmogus::refPavarde () [inline]
```

Definition at line 41 of file [studentai.h](#).

4.2.3.7 refVardas()

```
string & Zmogus::refVardas () [inline]
```

Definition at line 40 of file [studentai.h](#).

4.2.3.8 setPavarde()

```
void Zmogus::setPavarde (
    const string & p) [inline]
```

Definition at line 45 of file [studentai.h](#).

4.2.3.9 setVardas()

```
void Zmogus::setVardas (
    const string & v) [inline]
```

Definition at line 44 of file [studentai.h](#).

4.2.4 Member Data Documentation

4.2.4.1 pavarde

```
string Zmogus::pavarde [protected]
```

Definition at line 27 of file [studentai.h](#).

4.2.4.2 vardas

```
string Zmogus::vardas [protected]
```

Definition at line 26 of file [studentai.h](#).

The documentation for this class was generated from the following file:

- [studentai.h](#)

Chapter 5

File Documentation

5.1 funk.cpp File Reference

```
#include <iostream>
#include <vector>
#include <iomanip>
#include <string>
#include <cmath>
#include <algorithm>
#include <random>
#include <ctime>
#include <fstream>
#include <sstream>
#include <chrono>
#include "studentai.h"
#include <stdexcept>
```

Functions

- void **inputas** (vector< **Studentas** > &grupe)
rankinė studentų įvestis
- void **outputas** (const vector< **Studentas** > &grupe)
atspausdina studentus į console
- void **failoutputas** (const vector< **Studentas** > &grupe, string writing)
studentų spausdinimas į failą (be rikiavimo)
- void **randominputas** (vector< **Studentas** > &grupe, bool ArGeneruotiVardus)
atsitiktinių studentų generavimas
- void **failinputas** (vector< **Studentas** > &grupe, string reading)
studentų nuskaitymas iš failo
- string **randomvardas** (string lytis)
random vardo generavimas iš listo
- string **randompavarde** (string lytis)
random pavardės generavimas iš listo
- string **lytgen** ()
random lyties generavimas
- bool **cmpVardas** (const **Studentas** &a, const **Studentas** &b)

- vardo palyginimas*
- bool `cmpPavarde` (const `Studentas` &a, const `Studentas` &b)
- pavardės palyginimas*
- bool `cmpRez` (const `Studentas` &a, const `Studentas` &b)
- rezultatų palyginimas*
- bool `cmpMedrez` (const `Studentas` &a, const `Studentas` &b)
- medianos rezultatų palyginimas*
- bool `cmpEgz` (const `Studentas` &a, const `Studentas` &b)
- egzamino rezultatų palyginimas*
- void `rikiuoti` (vector< `Studentas` > &grupe, int pasirinkimas)
- studentų rikiavimas pagal cmp funkcijas*
- void `failgeneravimas` ()
- generuoja 5 failus su atsitiktiniais studentais*
- void `genirasymas` (vector< `Studentas` > &grupe, vector< `Studentas` > &protingi, vector< `Studentas` > &neprotingi, string writing_good, string writing_bad, int rik_pasirinkimas, `SkirstymoStrategija` strategija)
- skaito gen failus iš failgeneravimas funkcijos, rikiuoja ir paskirsto į naujus failus*
- void `skirstyti` (const vector< `Studentas` > &grupe, vector< `Studentas` > &protingi, vector< `Studentas` > &neprotingi)
- studentų skirstymas į grupes pagal rezultatus*
- void `logResults` (const string &container, int size, double read_t, double sort_t, double split_t)
- void `skirstyti2` (vector< `Studentas` > &grupe, vector< `Studentas` > &neprotingi)
- studentų skirstymas antra taktika*
- void `skirstyti3` (vector< `Studentas` > &grupe, vector< `Studentas` > &neprotingi)
- studentų skirstymas trečia taktika*
- ostream & `operator<<` (ostream &os, const `Studentas` &s)
- output operatorius klasei*
- istream & `operator>>` (istream &is, `Studentas` &s)
- input operatorius klasei*

5.1.1 Function Documentation

5.1.1.1 cmpEgz()

```
bool cmpEgz (
    const Studentas & a,
    const Studentas & b)
```

egzamino rezultatų palyginimas

Definition at line 363 of file `funk.cpp`.

5.1.1.2 cmpMedrez()

```
bool cmpMedrez (
    const Studentas & a,
    const Studentas & b)
```

medianos rezultatų palyginimas

Definition at line 358 of file `funk.cpp`.

5.1.1.3 cmpPavarde()

```
bool cmpPavarde (  
    const Studentas & a,  
    const Studentas & b)
```

pavardės palyginimas

Definition at line 348 of file [funk.cpp](#).

5.1.1.4 cmpRez()

```
bool cmpRez (  
    const Studentas & a,  
    const Studentas & b)
```

rezultatų palyginimas

Definition at line 353 of file [funk.cpp](#).

5.1.1.5 cmpVardas()

```
bool cmpVardas (  
    const Studentas & a,  
    const Studentas & b)
```

vardo palyginimas

Definition at line 343 of file [funk.cpp](#).

5.1.1.6 failgeneravimas()

```
void failgeneravimas ()
```

generuoja 5 failus su atsitiktiniais studentais

Definition at line 384 of file [funk.cpp](#).

5.1.1.7 failinputas()

```
void failinputas (  
    vector< Studentas > & grupe,  
    string reading)
```

studentų nuskaitymas iš failo

Definition at line 259 of file [funk.cpp](#).

5.1.1.8 failoutputas()

```
void failoutputas (  
    const vector< Studentas > & grupe,  
    string writing)
```

studentų spausdinimas į failą (be rikiavimo)

Write students to file

Definition at line 141 of file [funk.cpp](#).

5.1.1.9 genirasymas()

```
void genirasymas (  
    vector< Studentas > & grupe,  
    vector< Studentas > & protingi,  
    vector< Studentas > & neprotingi,  
    string writing_good,  
    string writing_bad,  
    int rik_pasirinkimas,  
    SkirstymoStrategija strategija)
```

skaito gen failus iš failgeneravimas funkcijos, rikiuoja ir paskirsto į naujus failus

Definition at line 418 of file [funk.cpp](#).

5.1.1.10 inputas()

```
void inputas (  
    vector< Studentas > & grupe)
```

rankinė studentų įvestis

Definition at line 38 of file [funk.cpp](#).

5.1.1.11 logResults()

```
void logResults (  
    const string & container,  
    int size,  
    double read_t,  
    double sort_t,  
    double split_t)
```

Definition at line 555 of file [funk.cpp](#).

5.1.1.12 lytgen()

```
string lytgen ()
```

random lyties generavimas

Definition at line 337 of file [funk.cpp](#).

5.1.1.13 operator<<()

```
ostream & operator<< (  
    ostream & os,  
    const Studentas & s)
```

output operatorius klasei

Definition at line 595 of file [funk.cpp](#).

5.1.1.14 operator>>()

```
istream & operator>> (  
    istream & is,  
    Studentas & s)
```

input operatorius klasei

Definition at line 604 of file [funk.cpp](#).

5.1.1.15 outputas()

```
void outputas (  
    const vector< Studentas > & grupe)
```

atspausdina studentus į console

Print students to console

Definition at line 120 of file [funk.cpp](#).

5.1.1.16 randominputas()

```
void randominputas (  
    vector< Studentas > & grupe,  
    bool ArGeneruotiVardus)
```

atsitiktinių studentų generavimas

Definition at line 166 of file [funk.cpp](#).

5.1.1.17 randompavarde()

```
string randompavarde (  
    string lytis)
```

random pavardės generavimas iš listo

Definition at line 324 of file [funk.cpp](#).

5.1.1.18 randomvardas()

```
string randomvardas (  
    string lytis)
```

random vardo generavimas iš listo

Definition at line 316 of file [funk.cpp](#).

5.1.1.19 rikiuoti()

```
void rikiuoti (  
    vector< Studentas > & grupe,  
    int pasirinkimas)
```

studentų rikiavimas pagal cmp funkcijas

Definition at line 369 of file [funk.cpp](#).

5.1.1.20 skirstyti()

```
void skirstyti (  
    const vector< Studentas > & grupe,  
    vector< Studentas > & protingi,  
    vector< Studentas > & neprotingi)
```

studentų skirstymas į grupes pagal rezultatus

Definition at line 543 of file [funk.cpp](#).

5.1.1.21 skirstyti2()

```
void skirstyti2 (  
    vector< Studentas > & grupe,  
    vector< Studentas > & neprotingi)
```

studentų skirstymas antra taktika

Definition at line 567 of file [funk.cpp](#).

5.1.1.22 skirstyti3()

```
void skirstyti3 (  
    vector< Studentas > & grupe,  
    vector< Studentas > & neprotingi)
```

studentų skirstymas trečia taktika

Definition at line 577 of file [funk.cpp](#).

5.2 funk.cpp

[Go to the documentation of this file.](#)

```

00001 #include <iostream>
00002 #include <vector>
00003 #include <iomanip>
00004 #include <string>
00005 #include <cmath>
00006 #include <algorithm>
00007 #include <random>
00008 #include <ctime>
00009 #include <fstream>
00010 #include <sstream>
00011 #include <chrono>
00012 #include "studentai.h"
00013 #include <stdexcept>
00014
00015 using namespace std;
00016
00017 void Studentas::skaiciuoti()
00018 {
00019     if (paz.empty()) return;
00020
00021     int sum = 0;
00022     for (int x : paz) sum += x;
00023
00024     rez = 0.4 * (sum * 1.0 / paz.size()) + 0.6 * egz;
00025
00026     vector<int> temp = paz;
00027     sort(temp.begin(), temp.end());
00028
00029     if (temp.size() % 2 == 0)
00030         medrez = (temp[temp.size()/2 - 1] + temp[temp.size()/2]) / 2.0;
00031     else
00032         medrez = temp[temp.size()/2];
00033
00034     medrez = medrez * 0.4 + egz * 0.6;
00035 }
00036
00037 void inputas(vector<Studentas> &grupe)
00038 {
00039     Studentas A;
00040
00041     while (true)
00042     {
00043         int sum = 0;
00044
00045         cout << "Vardas ir pavarde: ";
00046         cin >> A.refVardas() >> A.refPavarde();
00047
00048         int i = 0;
00049         string ats;
00050         while (true)
00051         {
00052             int temp;
00053             cout << "Iveskite pazymi numeris " << i + 1 << ": ";
00054             try
00055             {
00056                 cin >> temp;
00057                 if (cin.fail())
00058                     throw runtime_error("Bloga ivestis.");
00059             }
00060             catch (exception& e)
00061             {
00062                 cin.clear();
00063                 cin.ignore(10000, '\n');
00064                 cout << e.what() << " Bandykite dar karta.\n";
00065                 continue;
00066             }
00067
00068             A.refPaz().push_back(temp);
00069             sum += temp;
00070
00071             cout << "Ar toliau vesite pazymius? (y/n): ";
00072             cin >> ats;
00073             if (ats == "n") break;
00074             else if (ats == "y") cout << "Veskite toliau: ";
00075
00076             i++;
00077         }
00078
00079         cout << "Iveskite egzamino rezultata: ";
00080         try
00081         {
00082             cin >> A.refEgz();
00083         }

```

```

00084         if (cin.fail())
00085             throw runtime_error("Bloga ivestis.");
00086     }
00087     catch (exception& e)
00088     {
00089         cin.clear();
00090         cin.ignore(10000, '\n');
00091         cout << e.what() << " Bandykite dar karta.\n";
00092         continue;
00093     }
00094
00095     if (!A.refPaz().empty())
00096     {
00097         A.refRez() = (sum * 1.0 / A.refPaz().size()) * 0.4 + A.refEgz() * 0.6;
00098
00099         sort(A.refPaz().begin(), A.refPaz().end());
00100
00101         int n = A.refPaz().size();
00102         if (n % 2 == 0)
00103             A.refMedrez() = (A.refPaz()[n/2 - 1] + A.refPaz()[n/2]) / 2.0;
00104         else
00105             A.refMedrez() = A.refPaz()[n/2];
00106
00107         A.refMedrez() = A.refMedrez() * 0.4 + A.refEgz() * 0.6;
00108         A.skaiciuoti();
00109     }
00110
00111     grupe.push_back(A);
00112     A.clearPaz();
00113
00114     cout << "Ar toliau vesite mokinius? (y/n): ";
00115     cin >> ats;
00116     if (ats == "n") break;
00117 }
00118 }
00119
00120 void outputas(const vector<Studentas> &grupe)
00121 {
00122     cout << "-----" << endl;
00123     cout << left << setw(20) << "Vardas"
00124           << setw(20) << "Pavarde"
00125           << setw(20) << "Rezultatas (Vid.) /"
00126           << setw(20) << " Rezultatas (Med.)"
00127           << endl;
00128     cout << "-----" << endl;
00129
00130     for (const auto &A : grupe)
00131     {
00132         cout << left << setw(20) << A.getVardas()
00133               << setw(20) << A.getPavarde()
00134               << right << setw(20) << fixed << setprecision(2) << A.getRez()
00135               << setw(20) << A.getMedrez()
00136               << endl;
00137     }
00138 }
00139
00140
00141 void failoutputas(const vector<Studentas> &grupe, string writing)
00142 {
00143     ofstream out(writing);
00144     out << "----- \n";
00145     out << left << setw(20) << "Vardas"
00146           << setw(20) << "Pavarde"
00147           << setw(20) << "Rezultatas (Vid.) /"
00148           << setw(20) << " Rezultatas (Med.)"
00149           << "\n";
00150     out << "----- \n";
00151
00152     for (const Studentas& s : grupe)
00153     {
00154         out << left << setw(20) << s.getVardas()
00155               << setw(20) << s.getPavarde()
00156               << fixed << setprecision(2)
00157               << setw(20) << s.getRez()
00158               << setw(20) << s.getMedrez()
00159               << '\n';
00160     }
00161
00162     cout << "Studentu skaicius: " << grupe.size() << endl;
00163 }
00164
00165
00166 void randominputas(vector<Studentas> &grupe, bool ArGeneruotiVardus)
00167 {
00168     Studentas A;
00169     string lytis;
00170     string ats;
00171
00172     if (ArGeneruotiVardus)

```

```

00173     {
00174         int zmonsk = rand() % 100 + 1;
00175
00176         for(int iii = 0; iii < zmonsk; iii++)
00177         {
00178             int sum = 0;
00179
00180             lytis = lytgen();
00181             A.refVardas() = randomvardas(lytis);
00182             A.refPavarde() = randompavarde(lytis);
00183
00184             int pazsk = rand() % 100 + 1;
00185
00186             for (int ii = 0; ii < pazsk; ii++)
00187             {
00188                 int temp = rand() % 10 + 1;
00189                 A.refPaz().push_back(temp);
00190                 sum += temp;
00191             }
00192
00193             A.refEgz() = rand() % 10 + 1;
00194
00195             if (!A.refPaz().empty())
00196             {
00197                 A.refRez() = (sum * 1.0 / A.refPaz().size()) * 0.4 + A.refEgz() * 0.6;
00198
00199                 sort(A.refPaz().begin(), A.refPaz().end());
00200
00201                 int n = A.refPaz().size();
00202                 if (n % 2 == 0)
00203                     A.refMedrez() = (A.refPaz()[n/2 - 1] + A.refPaz()[n/2]) / 2.0;
00204                 else
00205                     A.refMedrez() = A.refPaz()[n/2];
00206
00207                 A.refMedrez() = A.refMedrez() * 0.4 + A.refEgz() * 0.6;
00208             }
00209
00210             grupe.push_back(A);
00211             A.clearPaz();
00212         }
00213     }
00214     else
00215     {
00216         while (true)
00217         {
00218             int sum = 0;
00219
00220             cout << "Vardas ir pavarde: ";
00221             cin >> A.refVardas() >> A.refPavarde();
00222
00223             int pazsk = rand() % 100 + 1;
00224
00225             for (int i = 0; i < pazsk; i++)
00226             {
00227                 int temp = rand() % 10 + 1;
00228                 A.refPaz().push_back(temp);
00229                 sum += temp;
00230             }
00231
00232             A.refEgz() = rand() % 10 + 1;
00233
00234             if (!A.refPaz().empty())
00235             {
00236                 A.refRez() = (sum * 1.0 / A.refPaz().size()) * 0.4 + A.refEgz() * 0.6;
00237
00238                 sort(A.refPaz().begin(), A.refPaz().end());
00239
00240                 int n = A.refPaz().size();
00241                 if (n % 2 == 0)
00242                     A.refMedrez() = (A.refPaz()[n/2 - 1] + A.refPaz()[n/2]) / 2.0;
00243                 else
00244                     A.refMedrez() = A.refPaz()[n/2];
00245
00246                 A.refMedrez() = A.refMedrez() * 0.4 + A.refEgz() * 0.6;
00247             }
00248
00249             grupe.push_back(A);
00250             A.clearPaz();
00251
00252             cout << "Ar toliau vesite zmones? (y/n): ";
00253             cin >> ats;
00254             if (ats == "n") break;
00255         }
00256     }
00257 }
00258
00259 void failinputas(vector<Studentas> &grupe, string reading)

```

```

00260 {
00261     ifstream file(reading);
00262     grupe.reserve(1000000);
00263
00264     if (!file)
00265     {
00266         cout << "Nepavyko atidaryti failo\n";
00267         return;
00268     }
00269
00270     string line;
00271     getline(file, line);
00272
00273     while (getline(file, line))
00274     {
00275         istringstream iss(line);
00276
00277         Studentas A;
00278
00279         iss >> A.refVardas() >> A.refPavarde();
00280
00281         int pazymys;
00282
00283         while (iss >> pazymys)
00284             A.refPaz().push_back(pazymys);
00285
00286         if (!A.refPaz().empty())
00287         {
00288             A.refEgz() = A.refPaz().back();
00289             A.refPaz().pop_back();
00290         }
00291
00292         if (!A.refPaz().empty())
00293         {
00294             int sum = 0;
00295             for (int x : A.refPaz())
00296                 sum += x;
00297
00298             sort(A.refPaz().begin(), A.refPaz().end());
00299
00300             int n = A.refPaz().size();
00301
00302             if (n % 2 == 0)
00303                 A.refMedrez() = (A.refPaz()[n/2 - 1] + A.refPaz()[n/2]) / 2.0;
00304             else
00305                 A.refMedrez() = A.refPaz()[n/2];
00306
00307             A.refMedrez() = A.refMedrez() * 0.4 + A.refEgz() * 0.6;
00308             A.refRez() = 0.4 * (sum * 1.0 / A.refPaz().size()) + 0.6 * A.refEgz();
00309         }
00310
00311         grupe.push_back(A);
00312     }
00313 }
00314
00315
00316 string randomvardas(string lytis)
00317 {
00318     if (lytis == "mot")
00319         return fvardai[rand() % fvardai.size()];
00320     else
00321         return mvardai[rand() % mvardai.size()];
00322 }
00323
00324 string randompavarde(string lytis)
00325 {
00326     int index = rand() % pavardes.size();
00327     string galune;
00328
00329     if (lytis == "mot")
00330         galune = fgalunes[rand() % fgalunes.size()];
00331     else
00332         galune = mgalunes[rand() % mgalunes.size()];
00333
00334     return pavardes[index] + galune;
00335 }
00336
00337 string lytgen()
00338 {
00339     return rand() % 2 == 0 ? "vyr" : "mot";
00340 }
00341
00342
00343 bool cmpVardas(const Studentas &a, const Studentas &b)
00344 {
00345     return a.getVardas() < b.getVardas();
00346 }

```



```

00347
00348 bool cmpPavarde(const Studentas &a, const Studentas &b)
00349 {
00350     return a.getPavarde() < b.getPavarde();
00351 }
00352
00353 bool cmpRez(const Studentas &a, const Studentas &b)
00354 {
00355     return a.getRez() < b.getRez();
00356 }
00357
00358 bool cmpMedrez(const Studentas &a, const Studentas &b)
00359 {
00360     return a.getMedrez() < b.getMedrez();
00361 }
00362
00363 bool cmpEgz(const Studentas &a, const Studentas &b)
00364 {
00365     return a.getEgz() < b.getEgz();
00366 }
00367
00368
00369 void rikiuoti(vector<Studentas> &grupe, int pasirinkimas)
00370 {
00371     if (pasirinkimas == 1)
00372         sort(grupe.begin(), grupe.end(), cmpVardas);
00373     else if (pasirinkimas == 2)
00374         sort(grupe.begin(), grupe.end(), cmpPavarde);
00375     else if (pasirinkimas == 3)
00376         sort(grupe.begin(), grupe.end(), cmpRez);
00377     else if (pasirinkimas == 4)
00378         sort(grupe.begin(), grupe.end(), cmpMedrez);
00379     else if (pasirinkimas == 5)
00380         sort(grupe.begin(), grupe.end(), cmpEgz);
00381 }
00382
00383
00384 void failgeneravimas()
00385 {
00386     int sk = 1000;
00387     for (int it = 0; it < 5; it++)
00388     {
00389         cout << "pradedamas " << sk << " eiluciu failo generavimo laiko skaiciavimas" << endl;
00390         auto input_start = chrono::high_resolution_clock::now();
00391
00392         ofstream gen("gen" + to_string(sk) + ".txt");
00393         gen << left << setw(20) << "STUD. VARDAS"
00394             << setw(20) << "STUD. PAVARDE"
00395             << setw(20) << "STUD. PAZYMIAI (PASKUTINIS EGZAMINO)\n";
00396
00397         for (int genit = 1; genit <= sk; genit++)
00398         {
00399             gen << left << setw(20) << "genVardas" + to_string(genit)
00400                 << setw(20) << "genPavarde" + to_string(genit);
00401
00402             for (int i = 1; i <= 16; i++)
00403                 gen << setw(20) << rand() % 10 + 1;
00404
00405             gen << "\n";
00406         }
00407
00408         gen.close();
00409         sk *= 10;
00410
00411         auto input_end = chrono::high_resolution_clock::now();
00412         chrono::duration<double> input_diff = input_end - input_start;
00413         cout << "Failu sukurimas + uzdarymas uztruko: " << input_diff.count() << " s \n";
00414     }
00415 }
00416
00417
00418 void genirasymas(vector<Studentas> &grupe, vector<Studentas> &protingi, vector<Studentas> &neprotingi,
00419 string writing_good, string writing_bad, int rik_pasirinkimas, SkirstymoStrategija strategija)
00420 {
00421     int sk = 1000;
00422
00423     for (int it = 0; it < 5; it++)
00424     {
00425         string filename = "gen" + to_string(sk) + ".txt";
00426
00427         cout << "Pradedamas failo nuskaitymas: " << filename << endl;
00428         auto start_read = chrono::high_resolution_clock::now();
00429
00430         ifstream file(filename);
00431
00432         if (!file)
00433     
```

```

00434         cout << "Nepavyko atidaryti failo\n";
00435         sk *= 10;
00436         continue;
00437     }
00438
00439     string line;
00440
00441     getline(file, line); // skip header
00442
00443     while (getline(file, line))
00444     {
00445         istringstream iss(line);
00446
00447         Studentas A;
00448
00449         iss >> A.refVardas() >> A.refPavarde();
00450
00451         int paz;
00452         while (iss >> paz)
00453             A.refPaz().push_back(paz);
00454
00455         if (A.refPaz().empty())
00456             continue;
00457
00458         A.refEgz() = A.refPaz().back();
00459         A.refPaz().pop_back();
00460
00461         int sum = 0;
00462         for (int x : A.refPaz())
00463             sum += x;
00464
00465         A.refRez() = 0.4 * (sum * 1.0 / A.refPaz().size()) + 0.6 * A.refEgz();
00466
00467         sort(A.refPaz().begin(), A.refPaz().end());
00468
00469         int n = A.refPaz().size();
00470
00471         if (n % 2 == 0)
00472             A.refMedrez() = (A.refPaz()[n/2 - 1] + A.refPaz()[n/2]) / 2.0;
00473         else
00474             A.refMedrez() = A.refPaz()[n/2];
00475
00476         A.refMedrez() = A.refMedrez() * 0.4 + A.refEgz() * 0.6;
00477
00478         grupe.push_back(A);
00479     }
00480
00481     file.close();
00482
00483     auto end_read = chrono::high_resolution_clock::now();
00484     chrono::duration<double> diff_read = end_read - start_read;
00485
00486     cout << "Failo nuskaitymas uztruko: " << diff_read.count() << " s\n";
00487
00488     auto start_rikiavimas = chrono::high_resolution_clock::now();
00489     rikiuoti(grupe, rik_pasirinkimas);
00490     auto end_rikiavimas = chrono::high_resolution_clock::now();
00491     chrono::duration<double> diff_rikiavimas = end_rikiavimas - start_rikiavimas;
00492
00493     cout << "Rikiavimas failui " << filename << " uztruko: " << diff_rikiavimas.count() << " s\n";
00494
00495     auto start_skirstymas = chrono::high_resolution_clock::now();
00496
00497     if (strategija == SkirstymoStrategija::PIRMAS)
00498         skirstyti(grupe, protingi, neprotingi);
00499     else if (strategija == SkirstymoStrategija::ANTRAS)
00500         skirstyti2(grupe, neprotingi);
00501     else if (strategija == SkirstymoStrategija::TRECIAS)
00502         skirstyti3(grupe, neprotingi);
00503
00504     auto end_skirstymas = chrono::high_resolution_clock::now();
00505     chrono::duration<double> diff_skirstymas = end_skirstymas - start_skirstymas;
00506
00507     cout << "Protingu/Neprotingu skirstymas failui " << filename << " uztruko: " <<
diff_skirstymas.count() << " s\n";
00508
00509     if (strategija == SkirstymoStrategija::PIRMAS)
00510     {
00511         cout << "Protingu ";
00512         failoutputas(protingi, "kursiokai_geri_" + to_string(sk) + ".txt");
00513         cout << "Neprotingu ";
00514         failoutputas(neprotingi, "kursiokai_blogi_" + to_string(sk) + ".txt");
00515     }
00516     else
00517     {
00518         cout << "Protingu ";
00519         failoutputas(grupe, "kursiokai_geri_" + to_string(sk) + ".txt");

```

```

00520         cout << "Neprotingu ";
00521         failoutputas(neprotingi, "kursiokai_blogi_" + to_string(sk) + ".txt");
00522     }
00523
00524     cout << "\n";
00525
00526     grupe.clear();
00527     protingi.clear();
00528     neprotingi.clear();
00529
00530     logResults(
00531         "vector",
00532         sk,
00533         diff_read.count(),
00534         diff_rikiavimas.count(),
00535         diff_skirstymas.count()
00536     );
00537
00538     sk *= 10;
00539 }
00540 }
00541
00542
00543 void skirstyti(const vector<Studentas>& grupe, vector<Studentas>& protingi, vector<Studentas>&
neprotingi)
00544 {
00545     for (const auto& s : grupe)
00546     {
00547         if (s.getRez() >= 5.0)
00548             protingi.push_back(s);
00549         else
00550             neprotingi.push_back(s);
00551     }
00552 }
00553
00554
00555 void logResults(const string& container, int size,
00556                 double read_t, double sort_t, double split_t)
00557 {
00558     ofstream out("results.csv", ios::app);
00559     out << container << ", "
00560         << size << ", "
00561         << read_t << ", "
00562         << sort_t << ", "
00563         << split_t << "\n";
00564 }
00565
00566
00567 void skirstyti2(vector<Studentas>& grupe, vector<Studentas>& neprotingi)
00568 {
00569     auto it = partition(grupe.begin(), grupe.end(),
00570         [](const Studentas& s) { return s.getRez() >= 5.0; });
00571
00572     neprotingi.assign(it, grupe.end());
00573     grupe.erase(it, grupe.end());
00574 }
00575
00576
00577 void skirstyti3(vector<Studentas>& grupe, vector<Studentas>& neprotingi)
00578 {
00579     neprotingi.reserve(grupe.size());
00580
00581     auto it = remove_if(grupe.begin(), grupe.end(),
00582         [&](const Studentas& s)
00583         {
00584             if (s.getRez() < 5.0)
00585             {
00586                 neprotingi.push_back(s);
00587                 return true;
00588             }
00589             return false;
00590         });
00591
00592     grupe.erase(it, grupe.end());
00593 }
00594
00595 ostream& operator<<(ostream& os, const Studentas& s)
00596 {
00597     os << left << setw(20) << s.getVardas()
00598         << setw(20) << s.getPavarde()
00599         << setw(10) << fixed << setprecision(2) << s.getRez()
00600         << setw(10) << s.getMedrez();
00601     return os;
00602 }
00603
00604 istream& operator>>(istream& is, Studentas& s)
00605 {

```

```

00606     s.refPaz().clear();
00607
00608     is » s.refVardas() » s.refPavarde();
00609
00610     int paz;
00611     while (is » paz) {
00612         s.refPaz().push_back(paz);
00613     }
00614
00615     if (!s.refPaz().empty()) {
00616         s.refEgz() = s.refPaz().back();
00617         s.refPaz().pop_back();
00618         s.skaiciuoti();
00619     }
00620
00621     return is;
00622 }

```

5.3 studentai.h File Reference

```

#include <iostream>
#include <vector>
#include <string>
#include <algorithm>

```

Classes

- class [Zmogus](#)
Bazinė abstrakti klasė [Zmogus](#).
- class [Studentas](#)
[Studentas](#) klasė, paveldi iš [Zmogus](#).

Enumerations

- enum class [SkirstymoStrategija](#) { [PIRMAS](#) , [ANTRAS](#) , [TRECIAS](#) }
Strategijos studentų skirstymui.

Functions

- void [inputas](#) (vector< [Studentas](#) > &grupe)
rankinė studentų įvestis
- void [failinputas](#) (vector< [Studentas](#) > &grupe, string reading)
studentų nuskaitymas iš failo
- void [randominputas](#) (vector< [Studentas](#) > &grupe, bool ArGeneruotiVardus)
atsitiktinių studentų generavimas
- void [outputas](#) (const vector< [Studentas](#) > &grupe)
atspausdina studentus į console
- void [failoutputas](#) (const vector< [Studentas](#) > &grupe, string writing)
studentų spausdinimas į failą (be rikiavimo)
- void [failgeneravimas](#) ()
generuoja 5 failus su atsitiktiniais studentais
- void [genirasymas](#) (vector< [Studentas](#) > &grupe, vector< [Studentas](#) > &protingi, vector< [Studentas](#) > &neprotingi, string writing_good, string writing_bad, int rik_pasirinkimas, [SkirstymoStrategija](#) strategija)
skaito gen failus iš failgeneravimas funkcijos, rikiuoja ir paskirsto į naujus failus

- string `lytgen` ()
random lyties generavimas
- string `randomvardas` (string lytis)
random vardo generavimas iš listo
- string `randompavarde` (string lytis)
random pavardės generavimas iš listo
- void `rikiuoti` (vector< `Studentas` > &grupe, int pasirinkimas)
studentų rikiavimas pagal cmp funkcijas
- bool `cmpVardas` (const `Studentas` &a, const `Studentas` &b)
vardo palyginimas
- bool `cmpPavarde` (const `Studentas` &a, const `Studentas` &b)
pavardės palyginimas
- bool `cmpRez` (const `Studentas` &a, const `Studentas` &b)
rezultatų palyginimas
- bool `cmpMedrez` (const `Studentas` &a, const `Studentas` &b)
medianos rezultatų palyginimas
- bool `cmpEgz` (const `Studentas` &a, const `Studentas` &b)
egzamino rezultatų palyginimas
- void `skirstyti` (const vector< `Studentas` > &grupe, vector< `Studentas` > &protingi, vector< `Studentas` > &neprotingi)
studentų skirstymas į grupes pagal rezultatus
- void `logResults` (const string &container, int size, double read_t, double sort_t, double split_t)
- void `skirstyti2` (vector< `Studentas` > &grupe, vector< `Studentas` > &neprotingi)
studentų skirstymas antra taktika
- void `skirstyti3` (vector< `Studentas` > &grupe, vector< `Studentas` > &neprotingi)
studentų skirstymas trečia taktika
- ostream & `operator<<` (ostream &os, const `Studentas` &s)
output operatorius klasei
- istream & `operator>>` (istream &is, `Studentas` &s)
input operatorius klasei

Variables

- const vector< string > `mvardai` = {"Vytenis", "Tomas", "Jonas", "Matas", "Simas", "Mantas", "Arnas"}
- const vector< string > `fvardai` = {"Egle", "Viktorija", "Vakare", "Inga", "Ema", "Marija", "Janina"}
- const vector< string > `pavardes` = {"Macerausk", "Jankausk", "Kazlausk", "Svilpausk", "Drugeliausk", "Briedausk"}
- const vector< string > `mgalunes` = {"as", "aitis"}
- const vector< string > `fgalunes` = {"iene", "aite", "yte"}
- const string `irasymo_failas` = "kursiokai.txt"
- const string `irasymo_failas_geras` = "kursiokai_geri.txt"
- const string `irasymo_failas_blogas` = "kursiokai_blogi.txt"

5.3.1 Enumeration Type Documentation

5.3.1.1 SkirstymoStrategija

```
enum class SkirstymoStrategija [strong]
```

Strategijos studentų skirstymui.

Enumerator

PIRMAS	
--------	--

ANTRAS	
TRECIAS	

Definition at line 17 of file [studentai.h](#).

5.3.2 Function Documentation

5.3.2.1 cmpEgz()

```
bool cmpEgz (
    const Studentas & a,
    const Studentas & b)
```

egzamino rezultatų palyginimas

Definition at line 363 of file [funk.cpp](#).

5.3.2.2 cmpMedrez()

```
bool cmpMedrez (
    const Studentas & a,
    const Studentas & b)
```

medianos rezultatų palyginimas

Definition at line 358 of file [funk.cpp](#).

5.3.2.3 cmpPavarde()

```
bool cmpPavarde (
    const Studentas & a,
    const Studentas & b)
```

pavardės palyginimas

Definition at line 348 of file [funk.cpp](#).

5.3.2.4 cmpRez()

```
bool cmpRez (
    const Studentas & a,
    const Studentas & b)
```

rezultatų palyginimas

Definition at line 353 of file [funk.cpp](#).

5.3.2.5 cmpVardas()

```
bool cmpVardas (
    const Studentas & a,
    const Studentas & b)
```

vardo palyginimas

Definition at line 343 of file [funk.cpp](#).

5.3.2.6 failgeneravimas()

```
void failgeneravimas ()
```

generuoja 5 failus su atsitiktiniais studentais

Definition at line 384 of file [funk.cpp](#).

5.3.2.7 failinputas()

```
void failinputas (
    vector< Studentas > & grupe,
    string reading)
```

studentų nuskaitymas iš failo

Definition at line 259 of file [funk.cpp](#).

5.3.2.8 failoutputas()

```
void failoutputas (
    const vector< Studentas > & grupe,
    string writing)
```

studentų spausdinimas į failą (be rikiavimo)

Write students to file

Definition at line 141 of file [funk.cpp](#).

5.3.2.9 genirasymas()

```
void genirasymas (
    vector< Studentas > & grupe,
    vector< Studentas > & protingi,
    vector< Studentas > & neprotingi,
    string writing_good,
    string writing_bad,
    int rik_pasirinkimas,
    SkirstymoStrategija strategija)
```

skaito gen failus iš failgeneravimas funkcijos, rikiuoja ir paskirsto į naujus failus

Definition at line 418 of file [funk.cpp](#).

5.3.2.10 inputas()

```
void inputas (  
    vector< Studentas > & grupe)
```

rankinė studentų įvestis

Definition at line 38 of file [funk.cpp](#).

5.3.2.11 logResults()

```
void logResults (  
    const string & container,  
    int size,  
    double read_t,  
    double sort_t,  
    double split_t)
```

Definition at line 555 of file [funk.cpp](#).

5.3.2.12 lytgen()

```
string lytgen ()
```

random lyties generavimas

Definition at line 337 of file [funk.cpp](#).

5.3.2.13 operator<<()

```
ostream & operator<< (  
    ostream & os,  
    const Studentas & s)
```

output operatorius klasei

Definition at line 595 of file [funk.cpp](#).

5.3.2.14 operator>>()

```
istream & operator>> (  
    istream & is,  
    Studentas & s)
```

input operatorius klasei

Definition at line 604 of file [funk.cpp](#).

5.3.2.15 outputas()

```
void outputas (
    const vector< Studentas > & grupe)
```

atspausdina studentus į console

Print students to console

Definition at line 120 of file [funk.cpp](#).

5.3.2.16 randominputas()

```
void randominputas (
    vector< Studentas > & grupe,
    bool ArGeneruotiVardus)
```

atsitiktinių studentų generavimas

Definition at line 166 of file [funk.cpp](#).

5.3.2.17 randompavarde()

```
string randompavarde (
    string lytis)
```

random pavardės generavimas iš listo

Definition at line 324 of file [funk.cpp](#).

5.3.2.18 randomvardas()

```
string randomvardas (
    string lytis)
```

random vardo generavimas iš listo

Definition at line 316 of file [funk.cpp](#).

5.3.2.19 rikiuoti()

```
void rikiuoti (
    vector< Studentas > & grupe,
    int pasirinkimas)
```

studentų rikiavimas pagal cmp funkcijas

Definition at line 369 of file [funk.cpp](#).

5.3.2.20 skirstyti()

```
void skirstyti (
    const vector< Studentas > & grupe,
    vector< Studentas > & protingi,
    vector< Studentas > & neprotingi)
```

studentų skirstymas į grupes pagal rezultatus

Definition at line 543 of file [funk.cpp](#).

5.3.2.21 skirstyti2()

```
void skirstyti2 (
    vector< Studentas > & grupe,
    vector< Studentas > & neprotingi)
```

studentų skirstymas antra taktika

Definition at line 567 of file [funk.cpp](#).

5.3.2.22 skirstyti3()

```
void skirstyti3 (
    vector< Studentas > & grupe,
    vector< Studentas > & neprotingi)
```

studentų skirstymas trečia taktika

Definition at line 577 of file [funk.cpp](#).

5.3.3 Variable Documentation

5.3.3.1 fgalunes

```
const vector<string> fgalunes = {"iene", "aite", "yte"}
```

Definition at line 15 of file [studentai.h](#).

5.3.3.2 fvardai

```
const vector<string> fvardai = {"Egle", "Viktorija", "Vakare", "Inga", "Ema", "Marija", "Janina"}
```

Definition at line 12 of file [studentai.h](#).

5.3.3.3 irasyimo_failas

```
const string irasyimo_failas = "kursiokai.txt"
```

Definition at line 159 of file [studentai.h](#).

5.3.3.4 irasymo_failas_blogas

```
const string irasymo_failas_blogas = "kursiokai_blogi.txt"
```

Definition at line 161 of file [studentai.h](#).

5.3.3.5 irasymo_failas_geras

```
const string irasymo_failas_geras = "kursiokai_geri.txt"
```

Definition at line 160 of file [studentai.h](#).

5.3.3.6 mgalunes

```
const vector<string> mgalunes = {"as", "aitis"}
```

Definition at line 14 of file [studentai.h](#).

5.3.3.7 mvardai

```
const vector<string> mvardai = {"Vytenis", "Tomas", "Jonas", "Matas", "Simas", "Mantas",  
"Arnas"}
```

Definition at line 11 of file [studentai.h](#).

5.3.3.8 pavardes

```
const vector<string> pavardes = {"Macerausk", "Jankausk", "Kazlausk", "Svilpausk", "Drugeliausk",  
"Briedausk"}
```

Definition at line 13 of file [studentai.h](#).

5.4 studentai.h

[Go to the documentation of this file.](#)

```
00001 #ifndef STUDENTAI_H
00002 #define STUDENTAI_H
00003
00004 #include <iostream>
00005 #include <vector>
00006 #include <string>
00007 #include <algorithm>
00008
00009 using namespace std;
00010
00011 const vector<string> mvardai = {"Vytenis", "Tomas", "Jonas", "Matas", "Simas", "Mantas", "Arnas"};
00012 const vector<string> fvardai = {"Egle", "Viktorija", "Vakare", "Inga", "Ema", "Marija", "Janina"};
00013 const vector<string> pavardes = {"Macerausk", "Jankausk", "Kazlausk", "Svilpausk", "Drugeliausk",  
"Briedausk"};
00014 const vector<string> mgalunes = {"as", "aitis"};
00015 const vector<string> fgalunes = {"iene", "aite", "yte"};
00016
00017 enum class SkirstymoStrategija
00018 {
00019     PIRMAS,
```

```

00020     ANTRAS,
00021     TRECIAS
00022 };
00024 class Zmogus {
00025 protected:
00026     string vardas;
00027     string pavarde;
00028
00029 public:
00031     Zmogus(const string& v = "A", const string& p = "BB")
00032     : vardas(v), pavarde(p)
00033     {}
00035     virtual ~Zmogus() {}
00036
00038     virtual void info() const = 0; // kad veiktu abstrakčiai
00039
00040     string& refVardas() { return vardas; }
00041     string& refPavarde() { return pavarde; }
00042     string getVardas() const { return vardas; }
00043     string getPavarde() const { return pavarde; }
00044     void setVardas(const string& v) { vardas = v; }
00045     void setPavarde(const string& p) { pavarde = p; }
00046
00048     Zmogus(const Zmogus&) = default;
00049     Zmogus& operator=(const Zmogus&) = default;
00050     Zmogus(Zmogus&&) noexcept = default;
00051     Zmogus& operator=(Zmogus&&) noexcept = default;
00052 };
00053
00055 class Studentas : public Zmogus {
00056 private:
00057     vector<int> paz;
00058     int egz = 0;
00059     double rez = 0.0;
00060     double medrez = 0.0;
00061
00062 public:
00064     Studentas(const string& v = "A",
00065               const string& p = "BB",
00066               const vector<int>& paz_ = {},
00067               int egz_ = 0)
00068     : Zmogus(), // zmogaus konstruktorius
00069       paz(paz_),
00070       egz(egz_),
00071       rez(0.0),
00072       medrez(0.0)
00073     {
00074         vardas = v;
00075         pavarde = p;
00076
00077         if (!paz.empty())
00078             skaiciuoti();
00079     }
00080
00081     ~Studentas()
00082     {
00083         paz.clear();
00084     }
00085
00086     // Rule of Five
00087
00088     Studentas(const Studentas& other)
00089     : Zmogus(other),
00090       paz(other.paz),
00091       egz(other.egz),
00092       rez(other.rez),
00093       medrez(other.medrez)
00094     {}
00095
00097     Studentas& operator=(const Studentas& other) {
00098         if (this != &other) {
00099             Zmogus::operator=(other);
00100             paz = other.paz;
00101             egz = other.egz;
00102             rez = other.rez;
00103             medrez = other.medrez;
00104         }
00105         return *this;
00106     }
00107
00109     Studentas(Studentas&& other) noexcept
00110     : Zmogus(std::move(other)),
00111       paz(std::move(other.paz)),
00112       egz(other.egz),
00113       rez(other.rez),
00114       medrez(other.medrez)
00115     {

```

```

00116     other.egz = 0;
00117     other.rez = 0;
00118     other.medrez = 0;
00119 }
00120
00122 Studentas& operator=(Studentas&& other) noexcept {
00123     if (this != &other) {
00124         Zmogus::operator=(std::move(other));
00125         paz = std::move(other.paz);
00126         egz = other.egz;
00127         rez = other.rez;
00128         medrez = other.medrez;
00129
00130         other.egz = 0;
00131         other.rez = 0;
00132         other.medrez = 0;
00133     }
00134     return *this;
00135 }
00136
00137 void info() const override { // kad veiktu ne abstrakciai?
00138     cout << "Studentas: " << vardas << " " << pavarde << endl;
00139 }
00140
00141 vector<int>& refPaz() { return paz; }
00142 int& refEgz() { return egz; }
00143 double& refRez() { return rez; }
00144 double& refMedrez() { return medrez; }
00145 void clearPaz() { paz.clear(); }
00146 double getRez() const { return rez; }
00147 double getMedrez() const { return medrez; }
00148 int getEgz() const { return egz; }
00149
00150 const vector<int>& getPaz() const { return paz; }
00151
00152 void addPaz(int p) { paz.push_back(p); }
00153 void setEgz(int e) { egz = e; }
00154
00155
00156 void skaiciuoti();
00157 };
00158
00159 const string irasyimo_failas = "kursiokai.txt";
00160 const string irasyimo_failas_geras = "kursiokai_geri.txt";
00161 const string irasyimo_failas_blogas = "kursiokai_blogi.txt";
00162
00163 void inputas(vector<Studentas> &grupe);
00164 void failinputas(vector<Studentas> &grupe, string reading);
00165 void randominputas(vector<Studentas> &grupe, bool ArGeneruotiVardus);
00166 void outputas(const vector<Studentas> &grupe);
00167 void failoutputas(const vector<Studentas> &grupe, string writing);
00168 void failgeneravimas();
00169 void genirasymas(vector<Studentas> &grupe, vector<Studentas> &protingi, vector<Studentas> &neprotingi,
00170     string writing_good, string writing_bad, int rik_pasirinkimas, SkirstymoStrategija strategija);
00171 string lytgen();
00172 string randomvardas(string lytis);
00173 string randompavarde(string lytis);
00174 void rikiuoti(vector<Studentas> &grupe, int pasirinkimas);
00175 bool cmpVardas(const Studentas &a, const Studentas &b);
00176 bool cmpPavarde(const Studentas &a, const Studentas &b);
00177 bool cmpRez(const Studentas &a, const Studentas &b);
00178 bool cmpMedrez(const Studentas &a, const Studentas &b);
00179 bool cmpEgz(const Studentas &a, const Studentas &b);
00180 void skirstyti(const vector<Studentas>& grupe, vector<Studentas>& protingi, vector<Studentas>&
    neprotingi);
00181 void logResults(const string& container, int size,
00182     double read_t, double sort_t, double split_t);
00183 void skirstyti2(vector<Studentas>& grupe, vector<Studentas>& neprotingi);
00184 void skirstyti3(vector<Studentas>& grupe, vector<Studentas>& neprotingi);
00185
00186 ostream& operator<<(ostream& os, const Studentas& s);
00187 istream& operator>>(istream& is, Studentas& s);
00188
00189
00190 #endif

```

5.5 versija_v2.0.cpp File Reference

```

#include <iostream>
#include <vector>

```

```
#include <iomanip>
#include <string>
#include <cmath>
#include <algorithm>
#include <random>
#include <ctime>
#include <fstream>
#include <sstream>
#include <chrono>
#include "studentai.h"
#include <stdexcept>
```

Functions

- `int main ()`

5.5.1 Function Documentation

5.5.1.1 main()

```
int main ()
```

Definition at line 17 of file [versija_v2.0.cpp](#).

5.6 versija_v2.0.cpp

[Go to the documentation of this file.](#)

```
00001 #include <iostream>
00002 #include <vector>
00003 #include <iomanip>
00004 #include <string>
00005 #include <cmath>
00006 #include <algorithm>
00007 #include <random>
00008 #include <ctime>
00009 #include <fstream>
00010 #include <sstream>
00011 #include <chrono>
00012 #include "studentai.h"
00013 #include <stdexcept>
00014
00015 using namespace std;
00016
00017 int main()
00018 {
00019     srand(time(NULL));
00020
00021     vector<Studentas> grupe;
00022     vector<Studentas> protingi;
00023     vector<Studentas> neprotingi;
00024
00025     grupe.reserve(10000000);
00026
00027     string genpas;
00028     cout << "Norite generuoti 5 failus? y/n " << endl;
00029     cin >> genpas;
00030     if (genpas == "y") failgeneravimas();
00031
00032
00033     /*
00034     cout << "--- Zmogus kurimo testavimas ---" << endl;
00035
```

```

00036 // Zmogus z;
00037
00038 cout << "--- Rule of Five dalykai ---" << endl;
00039
00040 Studentas a;
00041 a.setVardas("Jonas");
00042 a.setPavarde("Jonaitis");
00043 a.addPaz(10);
00044 a.addPaz(9);
00045 a.setEgz(8);
00046 a.skaiciuoti();
00047
00048 Studentas d(move(a)); // move constructor
00049 cout << setw(20) << "Move ctor (d inicializuojamas move'inant a): " << d << endl;
00050 cout << setw(20) << "objektas a, is kurio paimta info (a turi buti tuscias): " << endl;
00051 cout << a << endl;
00052
00053 Studentas e;
00054 e = move(b); // move assignment
00055 cout << setw(20) << "Move assign (e egzistuoja, b -> e): " << e << endl;
00056 b.addPaz(1);
00057 b.skaiciuoti();
00058 cout << setw(20) << "objektas b pakeiciamas" << endl;
00059 cout << setw(20) << "objektas b, is kurio paimta info: " << endl;
00060 cout << b << endl;
00061
00062 cout << setw(20) << "elementas e lygus elementui e ir yra move'inamas (e -> e): " << endl;
00063 e = e; // cia tas edge case kur move'ina i ta pati dalyka
00064 e = move(e);
00065 cout << e << endl;
00066
00067 cout << "=== End of Test ===" << endl << endl;
00068 */
00069
00070 int pasirinkimas = 4;
00071 string isvedimotipas, skaitymo_pasirinkimas, skaitymo_failas;
00072 cout << "Kokios norite ivesties? (1 - ranka arba is failo, 2 - generuoti tik pazymius, " << endl
00073 << " 3 - generuoti studentu vardus, pavardes ir pazymius, 4 - skaityti sugeneruotus failus; 5 arba
00074 kitas simbolis - baigti darba)" << endl;
00075 cin >> pasirinkimas;
00076
00077 int rikiavimo_pasirinkimas;
00078 cout << "Pagal ka rikiuoti?" << endl
00079 << "1 - vardas" << endl
00080 << "2 - pavarde" << endl
00081 << "3 - galutinis (vidurkis)" << endl
00082 << "4 - galutinis (mediana)" << endl
00083 << "5 - egzaminas" << endl;
00084 cin >> rikiavimo_pasirinkimas;
00085
00086 auto input_start = chrono::high_resolution_clock::now();
00087
00088 if (pasirinkimas == 1)
00089 {
00090     cout << "Ar skaityti duomenis is failo? (y - is failo / n - ranka);" << endl;
00091     cin >> skaitymo_pasirinkimas;
00092     if (skaitymo_pasirinkimas == "y")
00093     {
00094         string skaitymo_failas;
00095         cout << "Iveskite failo pavadinima: ";
00096         cin >> skaitymo_failas;
00097         failinputas(grupe, skaitymo_failas);
00098     }
00099     else inputas(grupe);
00100 }
00101 else if (pasirinkimas == 2)
00102 {
00103     randominputas(grupe, false);
00104 }
00105 else if (pasirinkimas == 3)
00106 {
00107     randominputas(grupe, true);
00108 }
00109 else if (pasirinkimas == 4)
00110 {
00111     genirasyimas(grupe, protingi, neprotingi, irasyimo_failas_geras, irasyimo_failas_blogas,
00112 rikiavimo_pasirinkimas, SkirstymoStrategija::TRECIAS);
00113     return 0;
00114 }
00115 else
00116 {
00117     cout << "darbas baigtas" << endl;
00118     return 0;
00119 }
00120 auto input_end = chrono::high_resolution_clock::now();

```

```
00121     chrono::duration<double> input_diff = input_end - input_start;
00122     cout << "Duomenų nuskaitymas užtruko: " << input_diff.count() << " s \n";
00123
00124     cout << "Ar rasyti į failą? (y/n)" << endl;
00125     cin >> isvedimotipas;
00126
00127     auto start = std::chrono::high_resolution_clock::now(); // Paleisti
00128
00129     rikiuoti(grupe, rikiavimo_pasirinkimas);
00130     //skirstyti(grupe, protingi, neprotingi)
00131     //skirstyti2(grupe, neprotingi);
00132     skirstyti3(grupe, neprotingi);
00133
00134     if (isvedimotipas == "y")
00135     {
00136         cout << "pradedu rasyti" << endl;
00137         failoutputas(protingi, "kursiokai_geri.txt");
00138         failoutputas(grupe, "kursiokai_geri_grupe.txt");
00139         failoutputas(neprotingi, "kursiokai_blogi.txt");
00140         cout << "Baigiu rasyti" << endl;
00141     }
00142     else
00143     {
00144         outputas(grupe);
00145     }
00146
00147     // is pavyzdzio
00148     auto end = chrono::high_resolution_clock::now(); // Stabdyti
00149     chrono::duration<double> diff = end-start; // Skirtumas (s)
00150     cout << "Programos rikiavimas ir isvedimas užtruko: " << diff.count() << " s \n";
00151
00152     cout << "program finished." << endl;
00153     return 0;
00154 }
```


Index

- ~Studentas
 - Studentas, [8](#)
- ~Zmogus
 - Zmogus, [13](#)
- addPaz
 - Studentas, [9](#)
- ANTRAS
 - studentai.h, [32](#)
- clearPaz
 - Studentas, [9](#)
- cmpEgz
 - funk.cpp, [18](#)
 - studentai.h, [32](#)
- cmpMedrez
 - funk.cpp, [18](#)
 - studentai.h, [32](#)
- cmpPavarde
 - funk.cpp, [18](#)
 - studentai.h, [32](#)
- cmpRez
 - funk.cpp, [19](#)
 - studentai.h, [32](#)
- cmpVardas
 - funk.cpp, [19](#)
 - studentai.h, [32](#)
- failgeneravimas
 - funk.cpp, [19](#)
 - studentai.h, [33](#)
- failinputas
 - funk.cpp, [19](#)
 - studentai.h, [33](#)
- failoutputas
 - funk.cpp, [19](#)
 - studentai.h, [33](#)
- fgalunes
 - studentai.h, [36](#)
- funk.cpp, [17](#)
 - cmpEgz, [18](#)
 - cmpMedrez, [18](#)
 - cmpPavarde, [18](#)
 - cmpRez, [19](#)
 - cmpVardas, [19](#)
 - failgeneravimas, [19](#)
 - failinputas, [19](#)
 - failoutputas, [19](#)
 - genirasymas, [20](#)
 - inputas, [20](#)
 - logResults, [20](#)
 - lytgen, [20](#)
 - operator<<, [20](#)
 - operator>>, [21](#)
 - outputas, [21](#)
 - randominputas, [21](#)
 - randompavarde, [21](#)
 - randomvardas, [21](#)
 - rikiuoti, [22](#)
 - skirstyti, [22](#)
 - skirstyti2, [22](#)
 - skirstyti3, [22](#)
- fvardai
 - studentai.h, [36](#)
- genirasymas
 - funk.cpp, [20](#)
 - studentai.h, [33](#)
- getEgz
 - Studentas, [9](#)
- getMedrez
 - Studentas, [9](#)
- getPavarde
 - Zmogus, [13](#)
- getPaz
 - Studentas, [10](#)
- getRez
 - Studentas, [10](#)
- getVardas
 - Zmogus, [13](#)
- info
 - Studentas, [10](#)
 - Zmogus, [13](#)
- inputas
 - funk.cpp, [20](#)
 - studentai.h, [33](#)
- irasymo_failas
 - studentai.h, [36](#)
- irasymo_failas_blogas
 - studentai.h, [36](#)
- irasymo_failas_geras
 - studentai.h, [37](#)
- logResults
 - funk.cpp, [20](#)
 - studentai.h, [34](#)
- lytgen
 - funk.cpp, [20](#)
 - studentai.h, [34](#)

- main
 - versija_v2.0.cpp, 40
- mgalunes
 - studentai.h, 37
- mvardai
 - studentai.h, 37
- operator<<
 - funk.cpp, 20
 - studentai.h, 34
- operator>>
 - funk.cpp, 21
 - studentai.h, 34
- operator=
 - Studentas, 10
 - Zmogus, 14
- outputas
 - funk.cpp, 21
 - studentai.h, 34
- pavarde
 - Zmogus, 15
- pavardes
 - studentai.h, 37
- PIRMAS
 - studentai.h, 31
- randominputas
 - funk.cpp, 21
 - studentai.h, 35
- randompavarde
 - funk.cpp, 21
 - studentai.h, 35
- randomvardas
 - funk.cpp, 21
 - studentai.h, 35
- refEgz
 - Studentas, 10
- refMedrez
 - Studentas, 11
- refPavarde
 - Zmogus, 14
- refPaz
 - Studentas, 11
- refRez
 - Studentas, 11
- refVardas
 - Zmogus, 14
- rikiuoti
 - funk.cpp, 22
 - studentai.h, 35
- setEgz
 - Studentas, 11
- setPavarde
 - Zmogus, 14
- setVardas
 - Zmogus, 14
- skaiciuoti
 - Studentas, 11
- SkirstymoStrategija
 - studentai.h, 31
- skirstyti
 - funk.cpp, 22
 - studentai.h, 35
- skirstyti2
 - funk.cpp, 22
 - studentai.h, 36
- skirstyti3
 - funk.cpp, 22
 - studentai.h, 36
- studentai.h, 30
 - ANTRAS, 32
 - cmpEgz, 32
 - cmpMedrez, 32
 - cmpPavarde, 32
 - cmpRez, 32
 - cmpVardas, 32
 - failgeneravimas, 33
 - failinputas, 33
 - failoutputas, 33
 - fgalunes, 36
 - fvardai, 36
 - genirasymas, 33
 - inputas, 33
 - irasymo_failas, 36
 - irasymo_failas_blogas, 36
 - irasymo_failas_geras, 37
 - logResults, 34
 - lytgen, 34
 - mgalunes, 37
 - mvardai, 37
 - operator<<, 34
 - operator>>, 34
 - outputas, 34
 - pavardes, 37
 - PIRMAS, 31
 - randominputas, 35
 - randompavarde, 35
 - randomvardas, 35
 - rikiuoti, 35
 - SkirstymoStrategija, 31
 - skirstyti, 35
 - skirstyti2, 36
 - skirstyti3, 36
 - TRECIAS, 32
- Studentas, 7
 - ~Studentas, 8
 - addPaz, 9
 - clearPaz, 9
 - getEgz, 9
 - getMedrez, 9
 - getPaz, 10
 - getRez, 10
 - info, 10
 - operator=, 10
 - refEgz, 10

- refMedrez, [11](#)
- refPaz, [11](#)
- refRez, [11](#)
- setEgz, [11](#)
- skaiciuoti, [11](#)
- Studentas, [8](#), [9](#)

TRECIAS

- studentai.h, [32](#)

vardas

- Zmogus, [15](#)

versija_v2.0.cpp, [39](#)

- main, [40](#)

Zmogus, [12](#)

- ~Zmogus, [13](#)
- getPavarde, [13](#)
- getVardas, [13](#)
- info, [13](#)
- operator=, [14](#)
- pavarde, [15](#)
- refPavarde, [14](#)
- refVardas, [14](#)
- setPavarde, [14](#)
- setVardas, [14](#)
- vardas, [15](#)
- Zmogus, [13](#)